Laboratório Funcional · Aula 12

Rodrigo Del Padre · RA 24042092-2 · Turma B

Gerado em 16/10/2026, 20:18:41 · tempo registrado no cronômetro: 44:35

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

1) `poe(lista, x)`: Viola o princípio fundamental da transparência referencial por produzir efeito colateral (side-effect), mutando diretamente a estrutura do objeto `lista` original na memória através de `append(x)` em vez de retornar uma nova coleção.
2) `sorteia()`: Não é determinística; invoca gerador de números pseudoaleatórios que depende do estado interno mutável da biblioteca `random`, devolvendo resultados distintos para chamadas consecutivas mesmo sem receber parâmetros.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Antes da avaliação de qualquer cálculo ou transformação, o console permanece em branco (saída vazia no início).

Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]

1: A-1 A-2
2: True
3: 8
4: 18
5: [4, 9, 16]
6: 29
```

```
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função `proximo_id("A")` viola a transparência referencial porque depende de um estado global mutável mantido fora de seu escopo (contador sequencial compartilhado). Isso compromete a estabilidade de testes unitários automatizados: como cada chamada altera o estado global, os testes passam a depender da ordem em que são executados, provocando falhas intermitentes caso o ambiente de teste não seja resetado a cada asserção.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

Em `compor(f, g)(4)`, executa-se primeiro a função interna g sobre a entrada 4, e em seguida a função f sobre o resultado intermediário, gerando 14. Já em `compor(g, f)(4)`, o fluxo é inverso: avalia-se primeiro f(4) e o resultado é repassado como argumento para g, totalizando 18. A composição funcional não possui comutatividade ($f \circ g \neq g \circ f$), logo a ordem das etapas no encadeamento produz saídas numéricas distintas.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

O método `notas.sort()` é imperativo e mutável, reordenando os elementos in-place na própria lista de origem (causando efeito colateral). Por outro lado, `sorted(notas)` segue estritamente o paradigma funcional: respeita o princípio da imutabilidade gerando e retornando uma nova coleção devidamente ordenada, mantendo a lista original inalterada.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

Retorna a cabeça (primeiro elemento) da lista: A

Previsão: (cdr '(A B C))

Retorna a cauda da lista (todos exceto o primeiro): (B C)

Previsão: (cons '(A B) '(C D))

Acopla o primeiro elemento como cabeça do segundo: ((A B) C D)

Saídas (original e sem o apóstrofo)

```
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B

22
14
25
8
negativo
Backtrace:
           5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
    191:35  4 (_ #f)
    191:35  3 (_ #f)
    196:27  2 (_ #f)
    223:20  1 (proc #<directory (guile-user) f564883bc80>)
In unknown file:
           0 (%resolve-variable (7 . A) #<directory (guile-user) f56...>)
```

```
ERROR: In procedure %resolve-variable:
Unbound variable: A
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo é a forma sintática abreviada do operador quote, cuja finalidade é suprimir a avaliação imediata da expressão, informando ao interpretador que `(A B C)` deve ser interpretado como dado literal. Sem o apóstrofo, o Scheme segue a regra de avaliação homoicônica tradicional de Lisp: trata o primeiro item (o símbolo A) como uma função/operador a ser executada com os argumentos B e C; como A não possuía ligação (binding) no escopo, disparou erro de variável não associada (Unbound variable).

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
Resultado: C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

Para n = 6: a recursão tradicional demandou 12 passos acumulando até 5 multiplicações pendentes na pilha (expressão expandida até 42 caracteres); na versão em cauda, concluiu em 8 passos diretos, sem manter operações pendentes no frame (expressão estável de até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

No Scheme funcionará com sucesso graças à otimização de chamada em cauda (TCO). No Python haverá interrupção por estouro do limite máximo de recursão da call stack.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
# Prática 3b - A mesma recursão em cauda, agora em Python
# Cole em https://onecompiler.com/python
def conta_ate(n, acc):
    if n == 0:
        return acc
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação

print(conta_ate(500, 0))
print(conta_ate(1_000_000, 0))
Exit 1
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
```

```
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, a pilha de execução (call stack) cresce linearmente em proporção a n, pois cada chamada precisa manter seu registro de ativação para calcular a multiplicação pendente após o desempilhamento. Na recursão em cauda, o estado intermediário acumulado é transportado no próprio parâmetro (acumulador), dispensando cálculos posteriores ao retorno e permitindo reutilizar o mesmo frame de pilha.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme implementa obrigatoriamente por especificação a otimização de chamada em cauda (Tail Call Optimization - TCO): ao constatar que a chamada recursiva é a última instrução da função, o compilador/interpretador substitui o frame de pilha corrente, executando a recursão em espaço de memória constante O(1) equivalente a um loop iterativo. Já o interpretador padrão de Python (CPython) não aplica TCO por decisão de design (para manter o rastreamento completo de stack trace na depuração) e impõe um limite estrito de profundidade de pilha (por padrão 1000 chamadas), resultando no `RecursionError`.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 0  (esperado 3)
soma-quadrados: 0  (esperado 14)
pares: ()  (esperado (2 4 6))

tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
; 1) quantos elementos a lista tem?   (tamanho '(a b c)) => 3
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

; 2) soma dos quadrados               (soma-quadrados '(1 2 3)) => 14
(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

; 3) só os números pares              (pares '(1 2 3 4 5 6)) => (2 4 6)   use even?
(define (pares lis)
  (cond
    ((null? lis) '())
    ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
    (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

O aspecto mais desafiador foi habituar-se ao encadeamento de chamadas recursivas puras sem variáveis mutáveis de controle e coordenar os parênteses prefixados do Lisp. O predicado `null?` foi empregado como caso base para detectar a lista vazia; a primitiva `car` extrai o elemento da cabeça da lista para cômputo ou filtragem de paridade; e `cdr` separa a cauda remanescente para alimentar a chamada recursiva seguinte.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

`take 5 squares` imprimirá os 5 primeiros quadrados perfeitos: [0, 1, 4, 9, 16]. A expressão `primeiro 42 (div 1 0)` retornará 42 sem lançar erro, pois o segundo parâmetro nunca chega a ser avaliado.

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

Time limit exceeded
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell utiliza avaliação preguiçosa (lazy evaluation / call-by-need): expressões são avaliadas somente quando seu valor é estritamente indispensável para produzir a saída no terminal. Por isso, a função `take 5` apenas consome cinco elementos da lista infinita. Da mesma forma, `primeiro 42 (div 1 0)` retorna de pronto o primeiro argumento; como o segundo valor jamais é referenciado, a divisão por zero nunca é executada pela CPU.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

A busca `member 16 squares` encerra imediatamente com `True` porque 16 é um quadrado perfeito encontrado na sequência. Já 17 não é quadrado perfeito; como a função `member` tradicional varre linearmente até achar o valor ou esgotar a lista, o Haskell continuou gerando quadrados infinitamente sem interromper o laço, estourando o tempo limite (Time Limit Exceeded). Como a sequência é monotonicamente crescente, a correção consiste em abortar a busca assim que o elemento corrente for maior que o valor buscado (ex.: usando `takeWhile (<= x)` ou cláusula `elem x (takeWhile (<= x) squares)`).

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

A chamada `mais10(5)` devolverá 15, e `[1, 2, 3].map(vezes(5))` resultará no array mapeado [5, 10, 15].

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Através do mecanismo de closure (fechamento léxico): a função retornada empacota e retém uma referência viva ao ambiente de escopo léxico em que foi instanciada, mantendo o valor do parâmetro 10 acessível mesmo após o encerramento do frame da função criadora.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

No Java `Function`, o método `f.compose(g)` executa primeiramente a função passada por parâmetro g e aplica seu retorno em f (f(g(x))), equivalendo perfeitamente ao `compor(f, g)` da Prática 1. O método `f.andThen(g)` atua de forma oposta: processa primeiro f e depois despacha o resultado para g (g(f(x))), invertendo a direção do encadeamento funcional.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
                       .filter(n -> n % 2 == 0)
                       .mapToInt(n -> n * n)
                       .sum();

        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter(aluno => aluno.nota >= 6)
  .map(aluno => aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

```
244
ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional em pipeline é substancialmente mais fácil e concisa de ler, expressando diretamente a intenção dos dados (filtragem, transformação e agregação) sem código boilerplate de laços e índices. Contudo, a abordagem imperativa com laço tradicional costuma ser mais amigável para depuração detalhada via breakpoints passo a passo para visualização de estados voláteis, enquanto no pipeline funcional é necessário recorrer a operações auxiliares como `.peek()`.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A sintaxe funcional declarativa (como a de Haskell) é mais intuitiva para expressar regras matemáticas abstratas e pipelines sem estado colateral. A sobrecarga de desempenho estimada em ~2× é amplamente admissível em sistemas corporativos, serviços web, pipelines analíticos e microsserviços de negócio onde segurança de concorrência, robustez tipográfica e clareza de manutenção têm prioridade máxima sobre economia de ciclos de CPU. Por outro lado, esse custo é impraticável em desenvolvimento de drivers, sistemas operacionais de tempo real estrito (RTOS), telecomunicações de baixa latência e motores gráficos de jogos 3D, onde a alocação de memória e o acesso direto ao hardware exigem previsibilidade e controle imperativo absoluto.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O aspecto mais marcante foi comprovar na prática como conceitos teoricamente abstratos, como avaliação preguiçosa e recursão em cauda (TCO), conseguem manipular sequências infinitas e um milhão de iterações sem esgotar a memória do sistema.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O equívoco tipográfico reside na instrução `b <- h` no contexto de filtro/guarda, na qual utilizou-se indevidamente a seta de extração de lista/mônada (`<-`) no lugar do operador relacional de comparação menor ou igual (`<=`). O compilador do Haskell acusou incompatibilidade de tipos porque esperava uma expressão estritamente booleana (`Bool`) para avaliar a condição do predicado, mas encontrou uma computação tipográfica desconexa decorrente da seta invertida.